Task1.1

编写代码编写代码的总体思路

实验的核心是模拟“用A编码，用B解码”的过程。在计算机中，文本在内存里是以统一的、抽象的Unicode码点存在的（例如，汉字“你”是`U+4F60`）。但当文本需要被存储或传输时，就必须通过一种具体的**编码（Encode）**规则将其转换为字节序列（Byte Stream）。当需要再次读取和显示文本时，则需要通过相应的**解码（Decode）**规则将字节序列转换回内存中的Unicode字符。

乱码的产生，正是解码规则与编码规则不匹配导致的。我的编程思路遵循以下步骤：

1. **定义基准**: 创建一个包含中文字符的原始文本字符串 `original_text` 作为我们所有实验的“标准答案”。

   ```
   original_text = "你好，这个世界，会出现乱码吗"
   ```

2. **正确编码**: 将此原始文本分别用 `utf-8` 和 `gbk` 两种编码方式转换成字节流（`utf8_bytes` 和 `gbk_bytes`）。

```
# 将原始文本编码为 UTF-8 和 GBK 字节流
utf8_bytes = original_text.encode('utf-8')
gbk_bytes = original_text.encode('gbk')
```

模拟乱码: 对这两种字节流，使用错误的解码方式进行解码。例如，用 `gbk` 解码器去读 `utf8_bytes`。

```
decoded_from_utf8_by_gbk_replaced = utf8_bytes.decode('gbk', errors='replace')
print(f"1. 用 GBK 解码 UTF-8: \n   {decoded_from_utf8_by_gbk_replaced}\n")
```

模拟恢复: 对于被判定为可恢复的乱码（即用 `latin-1` 生成的乱码），执行逆向操作。即将乱码字符串用当初错误的编码(`latin-1`)重新编码成字节流，会发现这个字节流与原始字节流完全相同。再用正确的解码器解码，即可恢复原文。

```
# --- 恢复情形 3 (latin-1 解码 UTF-8) ---
print(">>> 正在恢复情形3...")
# 1. 将乱码字符串用 latin-1 编码，恢复出原始的 UTF-8 字节流
original_utf8_bytes_recovered = decoded_from_utf8_by_latin1.encode('latin-1')
# 2. 用正确的 UTF-8 解码
recovered_text_3 = original_utf8_bytes_recovered.decode('utf-8')
```

验证: 使用 `assert` 语句来程序化地验证恢复后的文本是否与原始文本完全一致，确保恢复过程的正确性。

一开始遇到了问题

```
--------------------------------------------------
1. 用 GBK 解码 UTF-8 (replace错误策略):
   浣犲ソ锛屽浠鑲鑲

2. 用 UTF-8 解码 GBK:
   ××ä××××

3. 用 latin-1 解码 UTF-8:
   ä½ å¥½ï¼□ä¸□ç□□

4. 用 latin-1 解码 GBK:
   ÄãºÃ£¬ÊÀ½ç

---------------------------------------------------------------------------
UnicodeEncodeError                        Traceback (most recent call last)
/tmp/ipython-input-5-1072928747.py in <cell line: 0>()
     52 # 后续无论如何编码解码，丢失的信息都无法找回。
     53 garbled_step1_5 = utf8_bytes.decode('gbk', errors='replace')
---> 54 reencoded_bytes_5 = garbled_step1_5.encode('gbk') # 按照题意，用解码方式重新编码
     55 final_garbled_5 = reencoded_bytes_5.decode('utf-8', errors='replace')
     56 print(f"5. UTF-8 -> GBK -> UTF-8: \n   第一步乱码: {garbled_step1_5}\n   最终乱码: {final_garbled_5}\n")

UnicodeEncodeError: 'gbk' codec can't encode character '\ufffd' in position 7: illegal multibyte sequence
```

情形5中遇到的问题：

1. `garbled_step1_5 = utf8_bytes.decode('gbk', errors='replace')`
   - 这一步，程序尝试用 `GBK` 规则去理解 `UTF-8` 的字节流。
   - 当遇到 `UTF-8` 中某个汉字（例如“这”，字节为 `e8 bf 99`）的字节序列时，`GBK` 解码器无法理解，因为它不构成合法的 `GBK` 编码。
   - 因为我们设置了 `errors='replace'`，解码器没有直接崩溃，而是将这个无法理解的字节序列替换成了 Unicode 标准的“替换字符”：**`` (U+FFFD)**。
   - 所以，执行完这句代码后，得到的字符串 `garbled_step1_5` 中，真实地包含了 `` ` 这个字符。`浣犲ソ锛屽繖涓涓栬晫锛屽細鍥虹爱辩爨錫` 中间的那个问号菱形就是它。
2. `reencoded_bytes_5 = garbled_step1_5.encode('gbk')`
   - **这就是出错的根源**。现在，程序试图将包含 `` ` 字符的那个乱码字符串，重新用`GBK` 编码成字节流。
   - 但是，`GBK` 编码字符集中，根本就没有定义 `` ` (U+FFFD) 这个字符。`GBK 解码器不知道该用哪个字节或字节组合来表示它。
   - 因此，编码器在这里只能选择“罢工”，抛出 `UnicodeEncodeError`，：“`\ufffd`， `gbk` 码表中不存在，我无法编码。”

修正与完整复现

为了让实验能继续进行下去，观察乱码的最终形态，我们需要在编码时也指定一个错误处理策略，就像解码时一样。我们可以同样使用 `errors='replace'`，让编码器将无法编码的 `` ` 字符替换成一个`GBK`编码中存在的“替代品”，通常是一个普通的问号`?`(其字节为`0x3f` `)。

```
# -*- coding: utf-8 -*-
original_text = "你好，这个世界，会出现乱码吗"
# --- 编码准备 ---
utf8_bytes = original_text.encode('utf-8')
gbk_bytes = original_text.encode('gbk')
print(f"原始文本: {original_text}")
print("-" * 50)
```

```python
# --- 乱码复现 ---

# 情形 1 & 2
decoded_from_utf8_by_gbk_replaced = utf8_bytes.decode('gbk', errors='replace')
print(f"1. 用 GBK 解码 UTF-8: \n   {decoded_from_utf8_by_gbk_replaced}\n")

decoded_from_gbk_by_utf8 = gbk_bytes.decode('utf-8', errors='replace')
print(f"2. 用 UTF-8 解码 GBK: \n   {decoded_from_gbk_by_utf8}\n")

# 情形 3 & 4
decoded_from_utf8_by_latin1 = utf8_bytes.decode('latin-1')
print(f"3. 用 latin-1 解码 UTF-8: \n   {decoded_from_utf8_by_latin1}\n")

decoded_from_gbk_by_latin1 = gbk_bytes.decode('latin-1')
print(f"4. 用 latin-1 解码 GBK: \n   {decoded_from_gbk_by_latin1}\n")

# 情形 5
garbled_step1_5 = utf8_bytes.decode('gbk', errors='replace')
# 修正之处：在编码时也加入错误处理
reencoded_bytes_5 = garbled_step1_5.encode('gbk', errors='replace')
final_garbled_5 = reencoded_bytes_5.decode('utf-8', errors='replace')
print(f"5. UTF-8 -> GBK -> UTF-8 (修正后): \n   第一步乱码: {garbled_step1_5}\n   最终乱码:
{final_garbled_5}\n")


# 情形 6
garbled_step1_6 = gbk_bytes.decode('utf-8', errors='replace')
# UTF-8可以编码U+FFFD，所以这里不会出错，但最终解码GBK时会出现“锟斤拷”
reencoded_bytes_6 = garbled_step1_6.encode('utf-8')
final_garbled_6 = reencoded_bytes_6.decode('gbk', errors='replace')
print(f"6. GBK -> UTF-8 -> GBK: \n   第一步乱码: {garbled_step1_6}\n   最终乱码:
{final_garbled_6}\n")
```

```
原始文本: 你好，这个世界，会出现乱码吗
--------------------------------------------------
1. 用 GBK 解码 UTF-8:
   浣犳ソ锛岃繛涓€涓栨晠锛屾細鍑虹幇涓嶇爫鍚嗗
2. 用 UTF-8 解码 GBK:
   ××ã××××××××磬××××××××××××
3. 用 latin-1 解码 UTF-8:
   ä½ å¥½ï¼□è¿□ä¸ªä¸□ç□□ï¼□ä¼□å□ºç□°ä¹±ç □å□□
4. 用 latin-1 解码 GBK:
   ÄãºÃ£¬Õâ¸öÊÀ½ç£¬»á³öÏÖÂÒÂëÂð
5. UTF-8 -> GBK -> UTF-8 (修正后):
   第一步乱码: 浣犳ソ锛岃繛涓€涓栨晠锛屾細鍑虹幇涓嶇爫鍚嗗
   最终乱码: 你好，这?世界，会出现乱码?
6. GBK -> UTF-8 -> GBK:
   第一步乱码: ××ã××××××××磬××××××××××××
   最终乱码: 锟斤拷茫锟斤拷锟斤拷锟斤拷锟斤拷锟界，锟斤拷锟斤拷锟斤拷锟斤拷锟斤拷锟斤拷锟斤拷锟
```

问题回答

1. 哪些情况可以恢复，哪些不可以恢复?

在默认“解码之后同时用这种编码方式重新编码并保存”的前提下，数据是否可以恢复，取决于第一次错误解码的过程是否**有损**。

- **可以恢复的情况:**
  - **情形 3 (用 `latin-1` 解码 `UTF-8`): 可以恢复。** `latin-1` 编码的特殊之处在于它能表示每一个字节（0x00 - 0xFF）。当它解码一个 `UTF-8` 字节流时，它只是将每个字节值一一对应到一个 `latin-1` 字符。这个过程是完全可逆的、无损的。要恢复，只需将乱码字符串用 `latin-1` 重新编码得到原始的 `UTF-8` 字节流，再用 `UTF-8` 正确解码即可。给出代码

    ```
    # --- 恢复情形 3 (latin-1 解码 UTF-8) ---
    print(">>> 正在恢复情形3...")
    # 1. 将乱码字符串用 latin-1 编码，恢复出原始的 UTF-8 字节流
    original_utf8_bytes_recovered = decoded_from_utf8_by_latin1.encode('latin-1')
    # 2. 用正确的 UTF-8 解码
    recovered_text_3 = original_utf8_bytes_recovered.decode('utf-8')

    print(f"原始乱码: {decoded_from_utf8_by_latin1}")
    print(f"恢复文本: {recovered_text_3}")
    # 通过断言验证恢复是否成功
    assert recovered_text_3 == original_text
    print("情形3 恢复成功! \n")
    ```

  - **情形 4 (用 `latin-1` 解码 `GBK`): 可以恢复。** 原因同上，`latin-1` 保留了所有的原始 `GBK` 字节信息。恢复过程也是先用 `latin-1` 编码回字节流，再用 `GBK` 解码。给出代码

```python
# --- 恢复情形 4 (latin-1 解码 GBK) ---
print(">>> 正在恢复情形4...")
# 1. 将乱码字符串用 latin-1 编码，恢复出原始的 GBK 字节流
original_gbk_bytes_recovered = decoded_from_gbk_by_latin1.encode('latin-1')
# 2. 用正确的 GBK 解码
recovered_text_4 = original_gbk_bytes_recovered.decode('gbk')

print(f"原始乱码: {decoded_from_gbk_by_latin1}")
print(f"恢复文本: {recovered_text_4}")
# 通过断言验证恢复是否成功
assert recovered_text_4 == original_text
print("情形4 恢复成功！")
```

结果呈现

```
--------------------------------------------------
第二部分：恢复过程演示
--------------------------------------------------
>>> 正在恢复情形3...
原始乱码: ä½ å¥½ï¼□è¿□ä¸ªä¸□ç□□ï¼□ä¼□å□ºç□°ä¹±ç □å□□
恢复文本: 你好，这个世界，会出现乱码吗
情形3 恢复成功！

>>> 正在恢复情形4...
原始乱码: ÄãºÃ£¬Õâ¸öÊÀ½ç£¬»á³öÏÖÂÒÂëÂð
恢复文本: 你好，这个世界，会出现乱码吗
情形4 恢复成功！
```

- **不可以恢复的情况：**
  - **情形 1 (用 `GBK` 解码 `UTF-8`): 不可恢复。** `UTF-8` 的多字节序列（如中文的3个字节）在 `GBK` 的编码空间里很大概率是无效的。解码器会用一个特殊的替换字符（如 `?` 或 ``）来代替这些无法识别的字节序列。这个替换操作是**有损的**，原始的字节信息已经丢失，无法复原。
  - **情形 2 (用 `UTF-8` 解码 `GBK`): 不可恢复。** `GBK` 的双字节序列通常不符合 `UTF-8` 的编码规则（例如，字节 `0xC4` 在 `UTF-8` 中不是一个有效的起始字节）。因此，`UTF-8` 解码器同样会将其视为错误并用 `` 替换，造成信息丢失。
  - **情形 5 (先用 `GBK` 解码 `UTF-8`，再...): 不可恢复。** 第一步已经是不可恢复的了，信息已经丢失。
  - **情形 6 (先用 `UTF-8` 解码 `GBK`，再...): 不可恢复。** 同上，第一步就造成了不可逆的信息损失。

2. “锟斤拷”乱码的产生原因解释

“锟斤拷”这个经典乱码的产生，其核心在于 **Unicode替换字符 `` (U+FFFD)** 的反复错误编解码。

过程如下：

1. **第一步：产生替换字符`** 当一个程序试图将一个在目标编码中不存在的字符进行保存时（例如，将一个特殊的emoji表情符号保存到只支持 `GBK` 的数据库），或者在解码时遇到无法识别的字节序列时（如我们上面实验的情形1和2），它会用一个标准的“替换字符” `(其Unicode码点为 `U+FFFD`) 来代替这个无法处理的数据。

2. **第二步：保存为 `UTF-8` 编码** 这个包含 `的文本，如果被一个支持` UTF-8 `的系统或程序保存，那么` 字符本身就会被 `UTF-8` 编码规则转换成字节流。`U+FFFD` 的 `UTF-8` 编码是三个字节：**`0xEF 0xBF 0xBD`**。
3. **第三步：用 `GBK` 错误地打开** 现在，另一个程序或用户错误地认为这个文件是 `GBK` 编码的，并尝试用 `GBK` 解码器来读取它。此时，解码器看到了 `EF BF BD` 这样的字节序列。
4. **第四步：`GBK` 解码器的翻译** `GBK` 编码中，高位字节（`>0x80`）是双字节字符的开始。解码器开始工作：
    - 它读取第一个字节 `0xEF`，认为这是一个双字节字符的开始。
    - 它读取第二个字节 `0xBF`，与前一个字节配对，形成 `0xEFBF`。在 `GBK` 编码表中，`0xEFBF` 对应的汉字是“**锟**”。
    - 接着，它读取第三个字节 `0xBD`，也认为是一个双字节字符的开始。
    - 如果原文中有多个 `` ` `` 字符连在一起（字节流就是 ...EF BF BD EF BF BD... ），那么解码器会读取下一个字节 0xEF，与 0xBD 配对，形成 0xBDEF。在 GBK 编码表中，0xBDEF` 对应的汉字是“**斤**”。
    - 然后，它读取 `0xBF` 和 `0xBD`，配对成 `0xBFBD`。在 `GBK` 编码表中，`0xBFBD` 对应的汉字是“**拷**”。
    - 这个过程会一直重复，不断地将 `EF BF BD` 的序列错配成“锟斤拷”。

总结： "锟斤拷" 本质上是 **`UTF-8` 编码下的替换字符 `` ` `` 的字节流 (`EF BF BD`)，被错误地当成 `GBK` 编码进行解码所产生的特定结果**。这三个汉字之所以是它们，是由 `GBK` 的码表和 `U+FFFD` 的 `UTF-8` 字节表示共同决定的巧合。

task1.2

用vscode保存为gbk编码，再用utf-8解码打开得到

> 原理类似贝斯五十??贝斯二十??码其内?为喝彩声帕帕利马魁北克回音查理均匀帕帕回音祖鲁人狐步探戈?拉均匀朱丽叶朱丽叶魁北克德尔?祖鲁人威??高尔??高尔??埃克??线喝彩声奥斯卡埃克斯射线祖鲁人麦克回音均匀祖鲁人?拉德尔?酒店胜利者高尔夫球公斤胜利者麦克十一月回音胜利者威??拉??德尔塔公斤印度查理麦克阿尔法印度均匀奥斯卡喝彩声塞

中文代号与北约音标字母对应表：

中文	北约音标字母 (NATO)	代表字母
喝彩声 (hècǎishēng)	Bravo	B
帕帕 (pàpà)	Papa	P
利马 (lìmǎ)	Lima	L
魁北克 (kuíběikè)	Quebec	Q
回音 (huíyīn)	Echo	E
查理 (chálǐ)	Charlie	C
均匀 (jūnyún)	Uniform	U
祖鲁人 (zǔlǔrén)	Zulu	Z
狐步 (húbù)	Foxtrot	F
探戈 (tàngē)	Tango	T
塞拉 / 塞拉利昂	Sierra	S
朱丽叶 (zhūlìyè)	Juliett	J
德尔塔 (dé'ěrtǎ)	Delta	D
威士忌 (wēishìjì)	Whiskey	W
高尔夫 / 高尔夫球	Golf	G
埃克斯射线 (āikēsī shèxiàn)	X-ray	X
奥斯卡 (àosīkǎ)	Oscar	O
麦克 (màikè)	Mike	M
酒店 (jiǔdiàn)	Hotel	H
胜利者 (shènglìzhě)	Victor	V
公斤 (gōngjīn)	Kilo	K
十一月 (shíyīyuè)	November	N
印度 (yìndù)	India	I
阿尔法 (ā'ěrfǎ)	Alfa	A

得到字母序列为BPLQECUPEZFTSUJJQDZWGGXBOXZMEUZSDHVGKVMNEVWSDKICMAIUOB

其实是先做的这题，然后卡在这里去网上找了base26和base58，写代码把这几个排列组合了也解不出，所以放弃了（

2.2task1

三种核心GB系列编码方式

1. GB 2312-80 (国家标准)

- **全称**：《信息交换用汉字编码字符集-基本集》，是中国最早的汉字编码国家标准。
- **编码方式**：**双字节定长编码**。
- **核心思想**：可以想象一个巨大的二维表格，这个表格有94行（称为“区”）和94列（称为“位”）。一个汉字或符号的位置就由它的“区号”和“位号”唯一确定。
  - **区位码**：例如，“啊”字位于16区的01位，其“区位码”就是 `1601`。
  - **国标码/交换码**：为了在计算机中传输和处理，需要将区位码转换为实际的字节。转换规则是：**将区号和位号分别加上十六进制的 `0xA0`** (即十进制的160)。
    - `第一个字节 = 区号 + 0xA0`
    - `第二个字节 = 位号 + 0xA0`
    - 所以，“啊”字的国标码就是 `(0x10 + 0xA0)` `(0x01 + 0xA0)` = `0xB0A1`。
  - **特点**：
    - 所有字节的值都在 `0xA1` 到 `0xF7` 之间，高位都为1，巧妙地避开了与标准ASCII码（0x00-0x7F）的冲突，使得中英文可以混合处理。
    - 收录了约6700个汉字和约700个符号，在当时基本满足日常使用。但对于古汉语、部分人名、方言字等则无能为力。

2. GBK (国家标准扩展)

- **全称**：《汉字内码扩展规范》(K代表“扩展”)，它不是一个正式的国家标准，而是微软制定的一个技术规范，但因其广泛应用而成为事实标准。
- **编码方式**：**双字节定长编码**。
- **核心思想**：在完全保留GB 2312所有内容的基础上，利用其未使用的编码空间来容纳更多的字符。
  - **扩展方式**：GB 2312的两个字节都在 `0xA1` - `0xF7` 范围内。GBK打破了这个限制，极大地扩展了可用编码空间：
    - **第一个字节**：范围从 `0x81` 扩展到 `0xFE`。
    - **第二个字节**：范围从 `0x40` 扩展到 `0xFE`（不包括 `0x7F`）。
  - **特点**：
    - 收录了超过21000个汉字，包含了繁体字和日、韩汉字，基本解决了GB 2312字数不足的问题。
    - 它的设计完全以兼容GB 2312为前提。

3. GB 18030-2022 (现行国家强制标准)

- **全称**：《信息技术中文编码字符集》，是中国现行最新的、强制性的国家标准。
- **编码方式**：**变长编码（1、2或4字节）**。这是它与前两者最根本的区别。

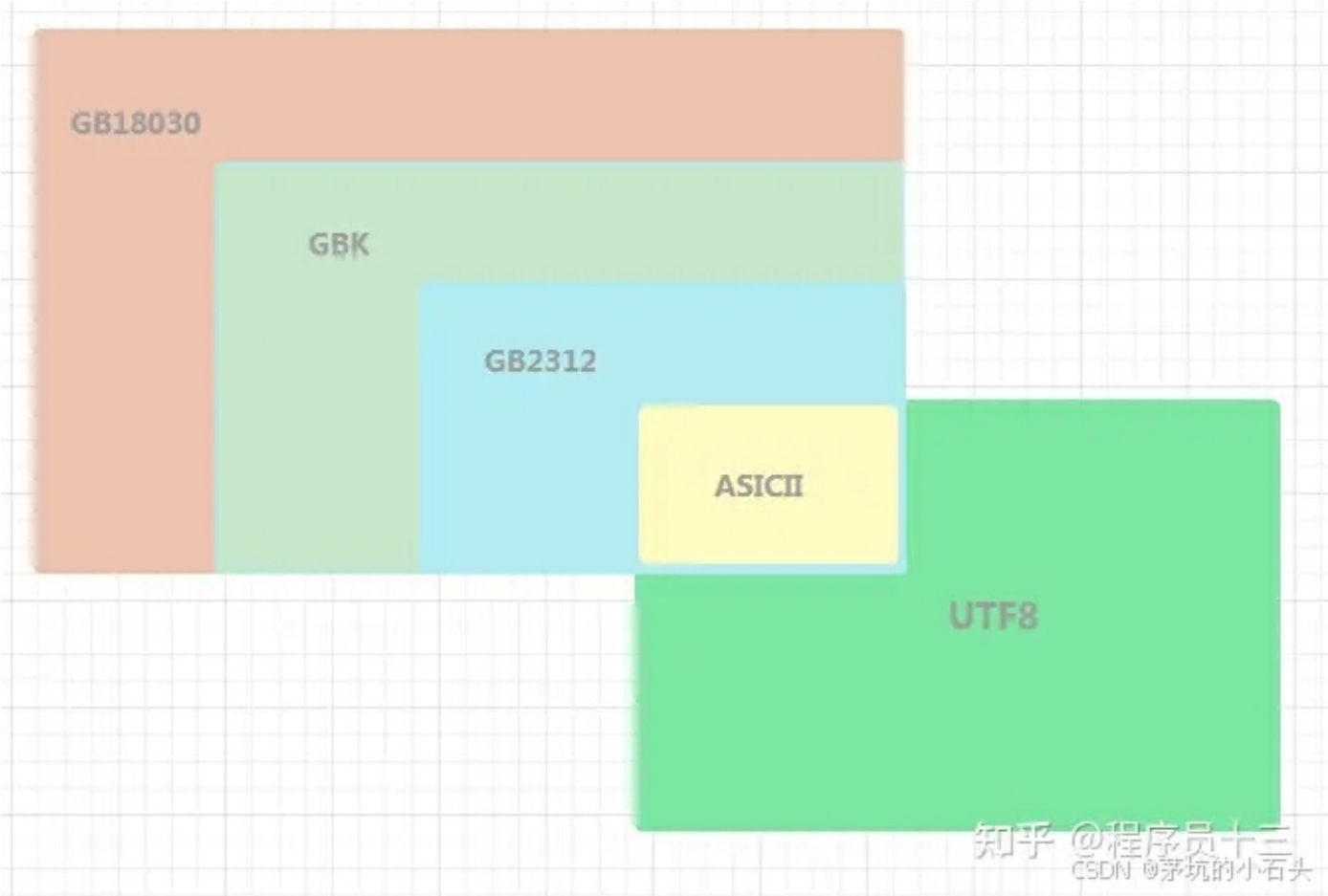
- **核心思想**：目标是收录Unicode标准中的所有字符，实现与国际标准的完全对接，同时保持对GBK和GB 2312的兼容。
  - **编码结构**：
    - **单字节**：与ASCII完全一致（`0x00` - `0x7F`）。
    - **双字节**：**完全重用GBK的编码方案**。所有在GBK中有效的双字节编码，在GB 18030中都代表同一个字符。
    - **四字节**：为映射Unicode中新增的大量字符而设计。其结构为 `[0x81–0xFE]` `[0x30–0x39]` `[0x81–0xFE]` `[0x30–0x39]`。解码器可以通过第二个字节是否在 `0x30`-`0x39` 范围内，来判断这是一个双字节字符还是四字节字符的开始。
  - **特点**：
    - 编码空间巨大，理论上可表示超过160万个字符。
    - 是GBK的超集，也是Unicode的实现方式之一（类似于UTF-8，但编码方式不同）。

GB系列三个版本的兼容性实现

GB系列编码的演进完美体现了“**向下兼容**”（Backward Compatibility）的设计哲学，即新版本必须能正确处理旧版本的数据。其实现方式可以概括为“**保留并扩展**”。

这就像盖房子：

- **GB 2312** 是最初建好的一栋7000个房间的**主楼**。
- **GBK** 没有拆除主楼，而是在主楼旁边**加盖了一栋更大的附楼**，并将两者连接起来。从GBK的视角看，GB 2312的主楼是其一部分。
- **GB 18030** 则是在GBK（主楼+附楼）的基础上，**又扩建了一片巨大的新园区**（四字节部分），同时确保了通往旧楼和附楼的道路（单字节ASCII和双字节GBK）完全畅通无阻。



具体的实现机制如下：

1. **编码空间的超集设计**：
    - **GBK 对 GB 2312 的兼容**：GBK的编码空间完全包含了GB 2312的编码空间。任何一个有效的GB 2312双字节编码（如`0xB0A1`），在GBK的规则下，也表示完全相同的字符。GBK只是利用了GB 2312未定义的字节范围（如第一个字节小于`0xA1`或第二个字节小于`0xA1`）来定义新字符，从而避免了冲突。
2. **编码结构的继承与区分**：
    - **GB 18030 对 GBK 的兼容**：GB 18030直接规定，其双字节部分的编码规则与GBK**完全相同**。
    - **区分机制**：GB 18030通过其变长编码的特性，可以明确区分不同长度的字符，从而实现共存。一个GB 18030解码器读取字节流时：
        - 如果字节在 `0x00-0x7F`，它知道这是一个单字节ASCII字符。
        - 如果字节在 `0x81-0xFE`，它知道这是一个多字节字符的开始，需要读取下一个字节。
        - 读取第二个字节后，如果它在 `0x40-0xFE` 范围，解码器就知道这是一个**双字节GBK字符**；如果它在 `0x30-0x39` 范围，解码器就知道这是一个**四字节字符**，并会继续读取后两个字节。

通过这种“**保留旧规则，增加新规则**”并确保新旧规则之间有明确区分的巧妙设计，GB系列编码成功地在近30年的发展中，从一个几千字的“基本集”演变成了一个能容纳上百万字符、与Unicode对齐的庞大体系，同时保证了历史数据的可用性和延续性。

2.2task2字

实验原理： 本次挑战是一个典型的多层解码谜题，其核心原理结合了两种技术：

1. **同形异义字隐写术 (Homoglyph Steganography):** 利用两种在视觉上极其相似、但在计算机内部编码（Unicode码位）完全不同的字符，来隐藏二进制信息（0和1）。这是信息藏匿的第一层。
2. **自定义二进制编码:** 从文本中提取出的原始二进制流并未采用如ASCII或UTF-8等标准编码，而是使用了一种为本题目量身定制的特殊编码方案。该方案的关键点在于非标准的分割长度（7位）、特定的数值变换和不常见的字符集（GBK）。这是信息解码的第二层。

实验步骤：

步骤一：从文本中提取隐藏的二进制流

此步骤对应step1.py，目的是完成隐写信息的提取。

1. **数据输入：** 首先，程序读取名为 `字.txt` 的文本文件。该文件包含一段长文本，其中混用了两种类型的汉字：
   - **普通汉字：** 来自标准的CJK（中日韩）统一表意文字区段。
   - **特殊汉字：** 来自康熙部首（Kangxi Radicals）区段，其外观与前者几乎无法用肉眼区分。
2. **建立映射规则：** 程序内置了两份字符列表（`normal_chars` 和 `special_chars`），并以此为依据建立了一个二进制映射规则：
   - 当在文本中遇到**普通汉字**时，将其映射为比特 `0`。
   - 当在文本中遇到**特殊汉字**时，将其映射为比特 `1`。
3. **信息提取：** 程序逐字遍历从文件中读取的全部文本内容。根据上一步建立的映射规则，将匹配到的字符逐一转换为 `0` 或 `1`，并按顺序拼接起来。
4. **初步输出：** 此步骤完成后，程序输出了一个由 `0` 和 `1` 组成的长字符串，即原始的、经过隐写处理的二进制信息流，以及它的总长度。

步骤二：解码二进制流，还原最终信息

此步骤对应step2.py，是对第一步得到的二进制流进行“解密”。

1. **数据分割：** 程序并非按照常规的8位（一字节）来分割二进制流，而是以 **7位** 为一个基本单元进行处理。这是解开谜题的第一个关键。
2. **终止符判断：** 程序在循环中检查每个7位单元。如果一个单元的二进制值是 `1111111`，则判定为信息结束的标志，解码过程立即停止。这说明二进制流的末尾部分可能不包含有效信息。
3. **数值变换：** 对于每一个有效的7位单元，程序执行了一个核心的数学变换。它将这个7位单元所代表的数值，加上一个固定的偏移量 `0b10100000`（即十进制的160，十六进制的`A0`）。
   - `新字节 = 原始7位数值 + 160`
4. **编码解码：** 经过数值变换后，得到一个新的字节（byte）序列。此序列并非ASCII或UTF-8编码。后使用了 `.decode("gbk")`，这些经过变换的字节需要使用**GBK编码**来进行最终解码。

实验结果： 第一步的代码获取二进制流和长度为374与题目提示符合，说明对了，然后再进行第二步的代码获得flag。

```
kaisenye@kaisendeMacBook-Air lab2 % python3 solve.py
成功从文件 '字.txt' 读取内容。
0000011010000100000110100001000001101000010000011101101100000110110101000001101111110000011101001000000110100101000
0110010100000001100100010000011010100100000110111001000001101111110000011100101100000110101110000001100100000000111
0101110000011011111100000110100111000001101000100000011101110100000000000000000000000000000000000000000000000000000
000000000000000000000000000 374
kaisenye@kaisendeMacBook-Air lab2 % python3 solve1.py
A A A {U _ r E 4 1 l Y _ k N 0 w _ G B} 牋牋牋牋牋牋
kaisenye@kaisendeMacBook-Air lab2 %
```

3.1

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	7760	100.0	6473364	200 k	0	0	0	7760
Malformed Packet	0.0	2	0.0	0	0	2	0	0	2
Ethernet	100.0	7758	1.7	108630	3369	0	0	0	7758
Internet Protocol Version 6	0.2	13	0.0	536	16	0	0	0	13
Internet Control Message Protocol v6	0.2	13	0.0	456	14	13	456	14	13
Internet Protocol Version 4	99.7	7739	2.4	154804	4802	0	0	0	7739
User Datagram Protocol	2.2	172	0.0	1376	42	0	0	0	172
Domain Name System	0.1	4	0.0	1028	31	4	1028	31	4
Datagram Transport Layer Security	2.2	168	0.8	51508	1597	123	51328	1592	168
Malformed Packet	0.6	45	0.0	0	0	45	0	0	45
Transmission Control Protocol	97.4	7557	3.9	249460	7738	1895	61068	1894	7557
Transport Layer Security	0.1	9	0.1	5807	180	9	3067	95	10
Hypertext Transfer Protocol	0.0	2	0.0	306	9	2	306	9	2
Data	72.8	5651	90.2	5842114	181 k	5651	5842114	181 k	5651
Internet Group Management Protocol	0.1	6	0.0	64	1	6	64	1	6
Internet Control Message Protocol	0.1	4	0.0	256	7	4	256	7	4
Address Resolution Protocol	0.1	6	0.0	168	5	6	168	5	6

Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Co
39.136.251.112	34	7 kB	15	5 kB	19	2 kB	
192.168.3.1	8	2 kB	4	1 kB	4	392 bytes	
192.168.3.73	2	108 bytes	2	108 bytes	0	0 bytes	
192.168.3.167	62	18 kB	31	16 kB	31	1 kB	
192.168.3.173	4	184 bytes	4	184 bytes	0	0 bytes	
192.168.3.208	7,697	6 MB	5,057	5 MB	2,640	1 MB	
192.168.3.209	7,592	6 MB	2,625	1 MB	4,967	5 MB	
192.168.3.255	71	32 kB	0	0 bytes	71	32 kB	
192.168.49.1	1	66 bytes	1	66 bytes	0	0 bytes	
192.168.49.3	1	66 bytes	0	0 bytes	1	66 bytes	
224.0.0.2	2	92 bytes	0	0 bytes	2	92 bytes	
224.0.0.22	2	108 bytes	0	0 bytes	2	108 bytes	
224.0.0.251	2	92 bytes	0	0 bytes	2	92 bytes	

协议分析（`Protocol Hierarchy`）:

- **97.4% 的流量是 TCP 协议。**
- 在 TCP 协议下，有一个巨大的 `Data` 部分（占总字节数的 90.2%）。这说明 Wireshark 无法识别跑在 TCP 上层的具体应用协议，这恰恰印证了这是厂商的私有协议（也就是我们正在寻找的软总线协议）。**所以我们的主战场将是 TCP 的数据部分。**

端点分析（`Endpoints`）:

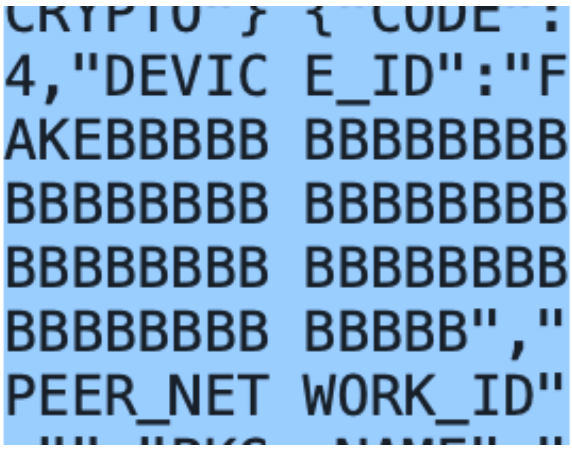
- 在所有 IP 地址中，`192.168.3.208` 和 `192.168.3.209` 这两个地址的流量占据了绝对主导地位（各自收发了约 7600 个包，总计 6MB 数据）。
- **这毫无疑问就是进行通信的两个目标设备。**

```
527  36644 → 45631 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=461 TSval=3726130520 TSecr=3295862158
595  36644 → 45631 [PSH, ACK] Seq=462 Ack=1 Win=64512 Len=529 TSval=3726130521 TSecr=3295862158
583  36644 → 45631 [PSH, ACK] Seq=991 Ack=1 Win=64512 Len=517 TSval=3726130523 TSecr=3295862158
577  36644 → 45631 [PSH, ACK] Seq=1508 Ack=1 Win=64512 Len=511 TSval=3726130524 TSecr=3295862158
691  36644 → 45631 [PSH, ACK] Seq=2019 Ack=1 Win=64512 Len=625 TSval=3726130526 TSecr=3295862158

, Ack: 1, Len: 46

mps

0000  08 fb ea 1a 6a 36 c0 f5  35 e4 0b 55 08 00 45 00   ····j6·· 5··U··E·
0010  02 01 cd 44 40 00 40 06  e2 c0 c0 a8 03 d0 c0 a8   ···D@·@· ····· ····
0020  03 d1 8f 24 b2 3f 7d 21  6d 3e 2a 1a f2 93 80 18   ···$·?}! m>*·····
0030  00 7e f7 95 00 00 01 01  08 0a de 18 3d 58 c4 72   ·~····· ····=X·r
0040  dd 8e 7b 22 72 65 71 75  65 73 74 49 64 22 3a 22   ··{"requ estId":"
0050  2d 38 38 38 31 39 38 30  31 34 30 32 32 32 36 38   -8881980 14022268
0060  38 32 36 22 2c 22 61 70  70 49 64 22 3a 22 66 61   826","ap pId":"fa
0070  6b 65 5f 73 6f 66 74 62  75 73 5f 61 75 74 68 22   ke_softb us_auth"
0080  2c 22 6d 73 67 22 3a 5b  7b 22 64 61 74 61 22 3a   ,"msg":[ {"data":
0090  7b 22 61 62 69 6c 69 74  79 22 3a 5b 7b 22 70 72   {"abilit y":[{"pr
00a0  6f 74 6f 63 6f 6c 22 3a  31 2c 22 74 64 43 6d 64   otocol": 1,"tdCmd
00b0  73 22 3a 5b 5d 7d 5d 2c  22 63 72 65 64 55 72 6c   s":[]}], "credUrl
00c0  22 3a 22 7b 5c 22 63 72  65 64 65 6e 74 69 61 6c   ":"{\"cr edential
00d0  54 79 70 65 5c 22 3a 30  2c 5c 22 6b 65 79 54 79   Type\":0 ,\"keyTy
00e0  70 65 5c 22 3a 31 2c 5c  22 74 72 75 73 74 54 79   pe\":1,\ "trustTy
00f0  70 65 5c 22 3a 31 2c 5c  22 69 73 44 69 72 65 63   pe\":1,\ "isDirec
0100  74 41 75 74 68 5c 22 3a  74 72 75 65 7d 22 2c 22   tAuth\": true}","
0110  73 70 43 6d 64 73 22 3a  5b 5d 2c 22 74 79 70 65   spCmds": [],"type
0120  22 3a 30 2c 22 76 72 22  3a 22 66 61 6b 65 2e 32   ":0,"vr" :"fake.2
0130  2e 30 2e 30 22 7d 2c 22  74 79 70 65 22 3a 32 7d   .0.0"},"  type":2}
0140  2c 7b 22 64 61 74 61 22  3a 7b 22 61 75 74 68 44   ,{"data" :{"authD
0150  61 74 61 22 3a 7b 22 46  41 4b 45 5f 46 49 45 4c   ata":{"F AKE_FIEL
0160  44 22 3a 22 66 6c 61 67  7b 63 62 35 61 37 64 62   D":"flag {cb5a7db
0170  37 35 39 38 36 30 38 34  63 35 33 63 38 65 30 61   75986084 c53c8e0a
0180  39 66 62 64 31 63 61 66  61 7d 22 2c 22 61 75 74   9fbd1caf a}","aut
0190  68 49 64 43 22 3a 22 34  32 44 39 46 33 31 37 43   hIdC":"4 2D9F317C
01a0  32 32 34 41 41 31 43 30  32 30 46 33 44 32 34 30   224AA1C0 20F3D240
01b0  41 30 33 42 44 42 44 30  30 22 2c 22 65 76 65 6e   A03BDBD0 0","even
01c0  74 22 3a 31 7d 2c 22 70  72 6f 74 6f 63 6f 6c 22   t":1},"p rotocol"
01d0  3a 31 7d 2c 22 74 79 70  65 22 3a 33 7d 5d 2c 22   :1},"typ e":3}],"
01e0  6f 70 43 6f 64 65 22 3a  30 2c 22 5f 46 41 4b 45   opCode": 0,"_FAKE
01f0  5f 4d 41 52 4b 45 52 22  3a 22 54 48 49 53 5f 49   _MARKER" :"THIS_I
0200  53 5f 46 41 4b 45 5f 43  52 59 50 54 4f 22 7d      S_FAKE_C RYPTO"}
```



带fake的内容这些应该是混淆的包内容，需要过滤

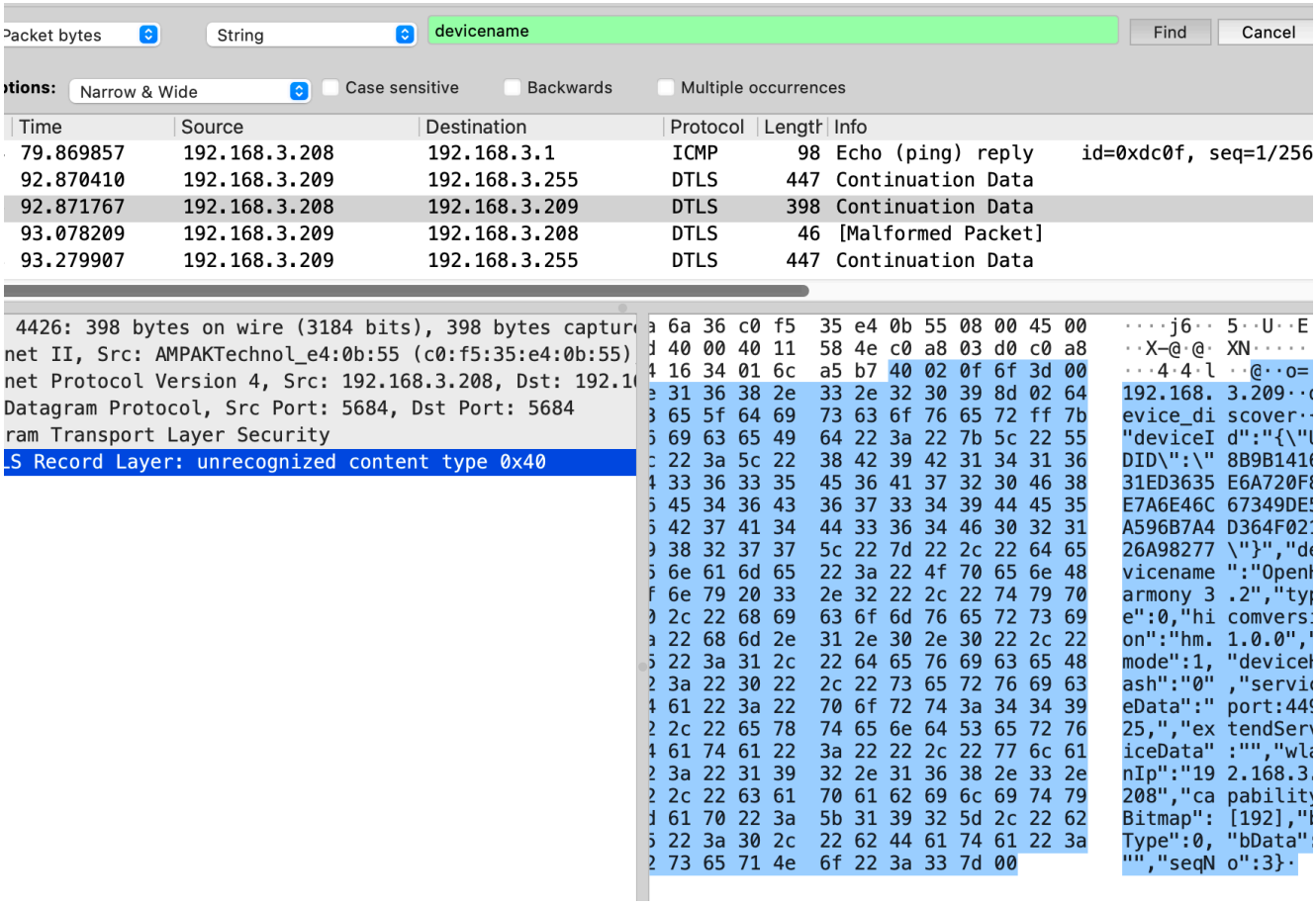
希望过滤带fake和FAKE的数据包。输入以下表达式

```
not (tcp.payload contains "fake" or tcp.payload contains "FAKE")
```

1. 设备名字 (Device Name)

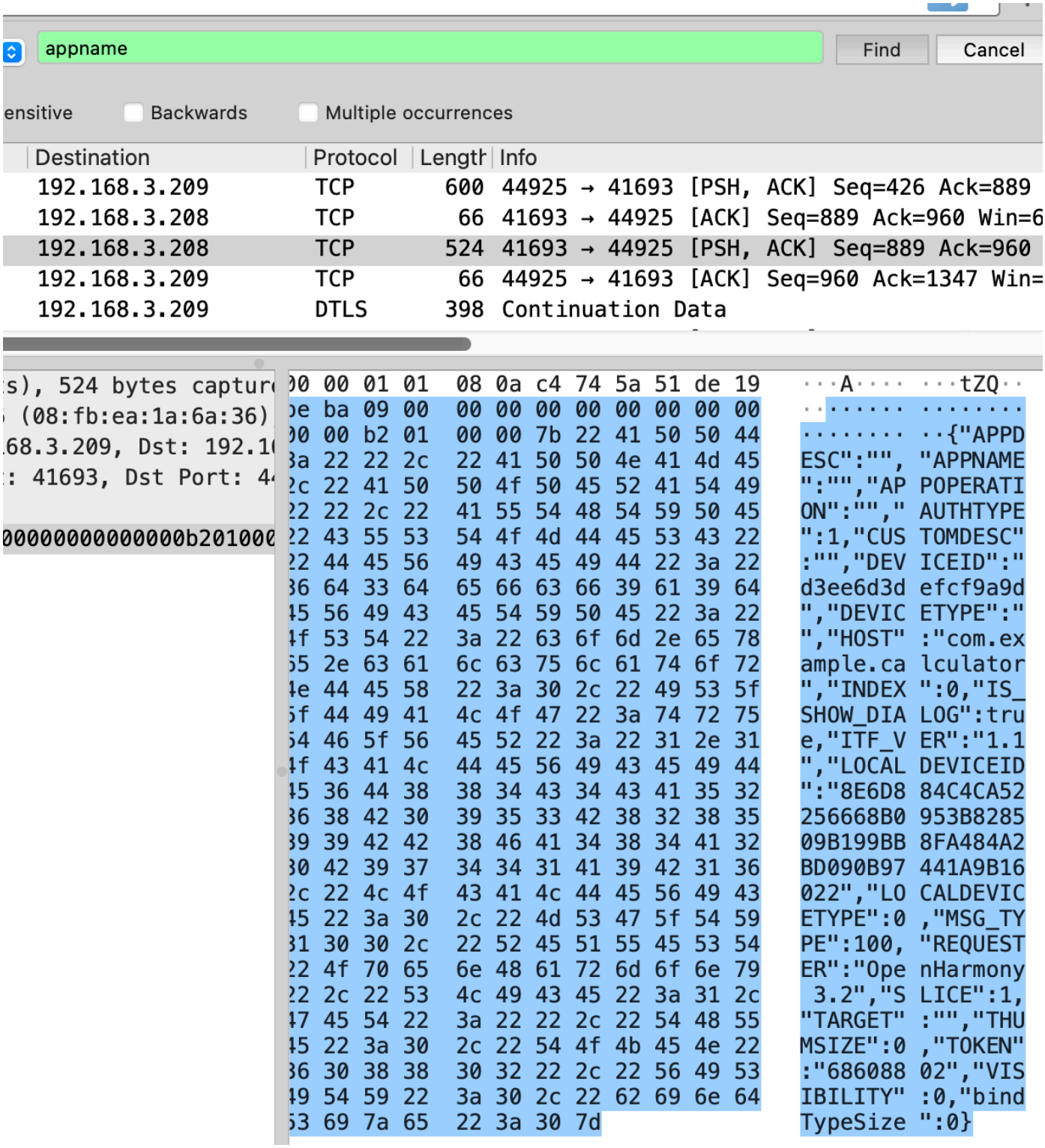
- **线索**: 题目提到“两个设备的名字是相同的”。

  截图可以看到



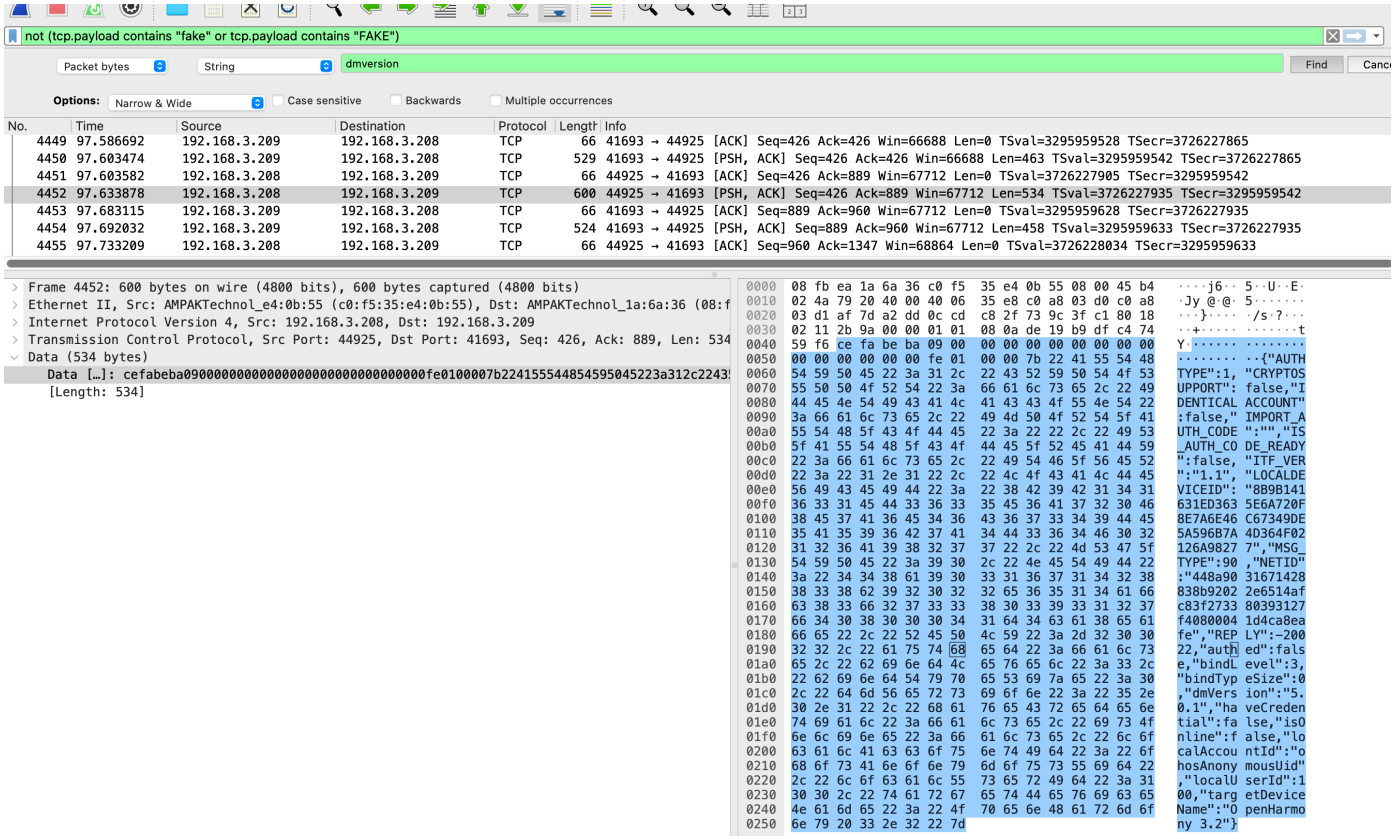
- **答案**: `OpenHarmony_3.2`

2. 应用程序 (Application)



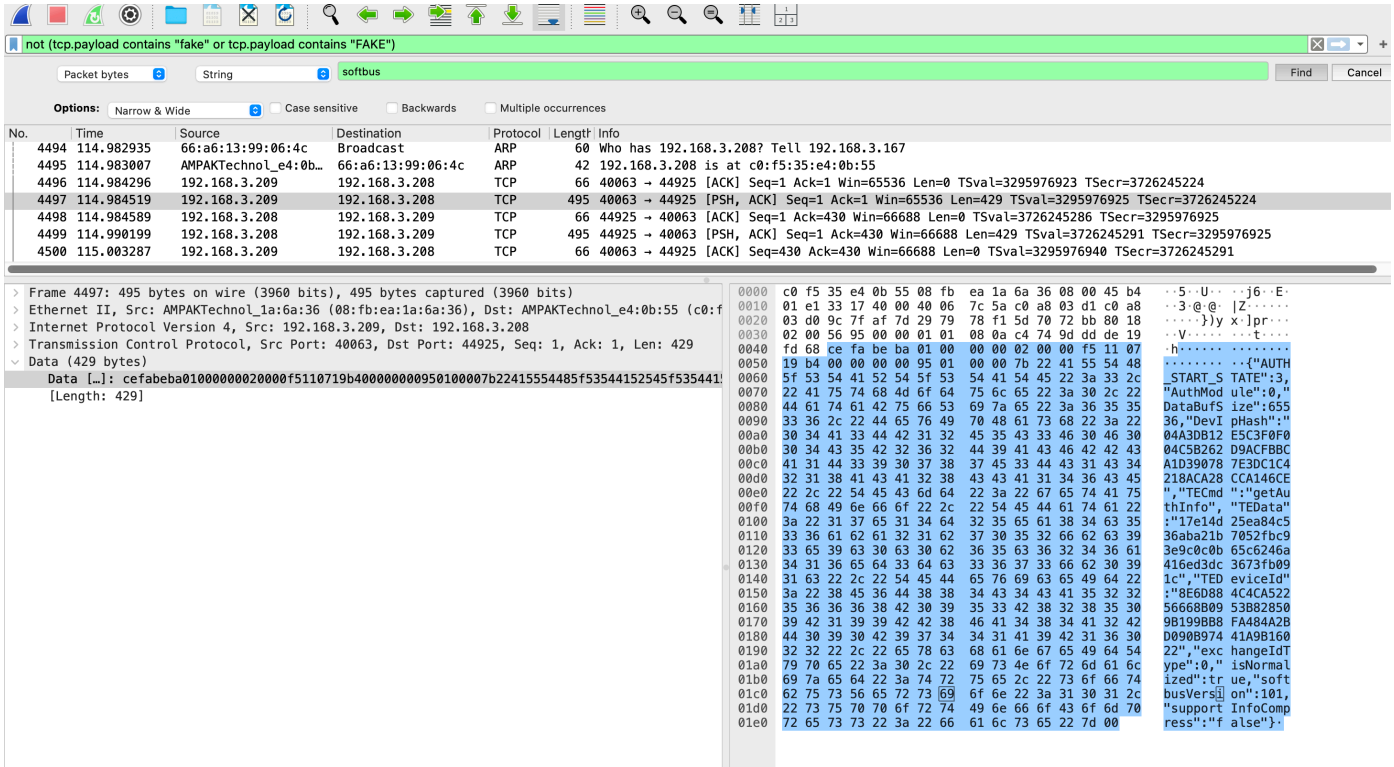
- 答案: calculator

3. 分布式设备管理组件版本号 (dmVersion)



- 答案: `5.0.1`

4. 软总线版本号 (Soft Bus Version)



- 答案: `101`

OpenHarmony_3.2_calculator_5.0.1_101转化为md5即可

[lab1 演示]软总线流量分析取证1 200 pts

题目类型： misc
题目难度： easy
题目作者： OpenHarmony CTF 2025
题目分类：misc lab 1

根据交互的流量包，回答以下问题:

1. 已知两个设备的名字是相同的，它们的名字是什么？（如果设备名中有空格则用_替代，例如Name 1.0写为Name_1.0）
2. `suspicious_traffic.pcap` 流量所对应的应用程序是什么？（英文名词单词，如application）
3. 目标设备的分布式设备管理组件版本号是什么？(如1.1.1)
4. 通信过程使用的软总线版本号是多少？(如999)

请将回答的四个问题的答案用下划线连接，进行md5加密后包裹上flag{}进行提交

该题目已被解出

bonus zjuwlan

镜像文件提取

- 收到 `godspeed.img` 文件，判断其为磁盘镜像而非图片文件。
- 使用 `7z x godspeed.img` 命令，成功从镜像中提取出文件内容。
- 提取后，发现两个关键目标：网络流量包 `crack_zju-01.cap` 和一个隐藏的 `Vim` 交换文件 `.password.txt.swp`。

```
kaisenye@kaisendeMacBook-Air zjuwlan % ls
crack_zju-01.cap        godspeed.img        _        lost+found
```

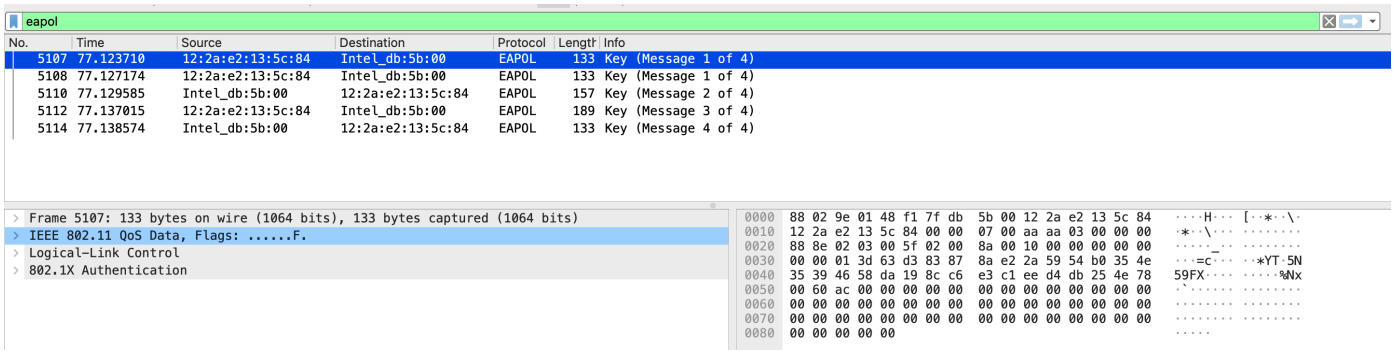
关键线索挖掘

- 识别出 `.password.txt.swp` 是由于编辑器非正常退出而遗留的信息泄露文件。

- 执行 `vim -r .password.txt.swp` 命令，成功恢复了交换文件中包含的原始文本。
- 将恢复的内容另存为新的文本文件 `mima.txt`，作为下一步破解所需的专用密码字典。

网络流量破解

- 通过 `Wireshark` 初步分析 `crack_zju-01.cap`，确认其中包含一个完整的 WPA 四次握手（EAPOL 报文），明确了破解目标。



- 调用 `aircrack-ng` 工具，并指定上一步获取的 `mima.txt` 文件作为密码字典，对握手包发起破解。
- 执行核心破解命令：`aircrack-ng crack_zju-01.cap -w mima.txt`。

```
kaisenye@kaisendeMacBook-Air zjuwlan % vim -r .password.txt.swp
kaisenye@kaisendeMacBook-Air zjuwlan % cat mima.txt
KpE4suAkq8
SJyhqPKqHR
RRIc3BgApI
yYQUDGereQ
```

```
kaisenye@kaisendeMacBook-Air zjuwlan % aircrack-ng crack_zju-01.cap -w mima.txt
Reading packets, please wait...
Opening crack_zju-01.cap
Read 6716 packets.

   #  BSSID              ESSID                     Encryption

   1  12:2A:E2:13:5C:84  ZJUWLAN                   WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening crack_zju-01.cap
Read 6716 packets.

1 potential targets



                              Aircrack-ng 1.7

      [00:00:00] 105/401 keys tested (2680.90 k/s)

      Time left: 0 seconds                                      26.18%

                        KEY FOUND! [ 0YcWPeLMBp ]


      Master Key     : 2A 0A 56 2C 6E 44 73 96 60 DB 3B F2 D5 76 9F 1A
                       E4 CD 5B C1 9A 08 62 FA EF 0F 65 E2 34 B4 D1 ED

      Transient Key  : 7B 14 E7 35 D7 3A 78 D0 E1 A6 45 FE FA 99 12 C0
```

实验结果

`aircrack-ng` 工具利用提供的密码字典 `mima.txt` 迅速完成了破解，成功找到了 WPA 网络的密钥。

- **最终密钥:** `0YcWPeLMBp`
- **对应 Flag :** `AAA{0YcWPeLMBp}`

suggestions

建议NATO26任务增加提示/中间成果检验，比如给一个程序检验初步根据音标对应得出的字母串是否正确，这个题感觉有点阴。。。QAQ